

## P2 SM

Autor 1:

- Apellidos, nombre: Brea Arias, Ismael
- Email UDC: ismael.brea@udc.es

Autor 2:

- Apellidos, nombre: González Ouzounis, Ruben
- Email UDC: ruben.gonzalez.ouzounis@udc.es

### Ejercicio 1: Configurar los clientes sip1 y sip2

Configura los clientes SIP **sip1** y **sip2** en Asterisk usando el fichero de configuración **pjsip\_custom.conf** del código **1**. Una vez configurados, conecta los clientes y ejecuta en la consola de Asterisk el comando **pjsip show endpoints**. **Copia la salida** y en base a esta responde a las siguientes preguntas (**justifica** las respuestas):

1. ¿Cual es el estado de los *endpoints* **sip1** y **sip2**?
2. ¿En que direcciones IP y puertos se puede contactar con estos *endpoints*?
3. ¿Cuántos contactos podrían registrarse como máximo para cada uno de los *endpoints*? ¿Cómo lo sabes?

```
freepbx*CLI> pjsip show endpoints

Endpoint: <Endpoint/CID.....> <State.....> <Channels.>
  I/OAuth: <AuthId/UserName.....>
  Aor: <Aor.....> <MaxContact>
  Contact: <Aor/ContactUri.....> <Hash.....> <Status> <RTT(ms)...>
  Transport: <TransportId.....> <Type> <cos> <tos> <BindAddress.....>
  Identify: <Identify/Endpoint.....>
  Match: <criteria.....>
  Channel: <ChannelId.....> <State.....> <Time.....>
  Exten: <DialedExten.....> CLCID: <ConnectedLineCID.....>
=====

Endpoint: sip1                                Not in use    0 of inf
  InAuth: authsip1/sip1
  Aor: sip1                                    1
  Contact: sip1/sip:sip1@10.20.24.137:42199;transport 6b940844bc NonQual  nan
  Transport: transport_udp                      udp          0          0 0.0.0.0:5060

Endpoint: sip2                                Not in use    0 of inf
  InAuth: authsip2/sip2
  Aor: sip2                                    1
  Contact: sip2/sip:sip2@10.20.28.176;transport=udp a0399a9dea NonQual  nan
  Transport: transport_udp                      udp          0          0 0.0.0.0:5060

Objects found: 2

freepbx*CLI>
```

1. ¿Cual es el estado de los endpoints sip1 y sip2?

Not in use para sip1 y sip2

2. ¿En que direcciones IP y puertos se puede contactar con estos endpoints?

sip1:

Dirección IP: 10.20.24.137

Puerto: 42199

sip2:

Dirección IP: 10.20.28.176

Puerto: No especificado explícitamente, por lo que se asume que usa el puerto por defecto del transporte UDP (5060).

3. ¿Cuántos contactos podrían registrarse como máximo para cada uno de los endpoints? ¿Cómo lo sabes?

En el archivo de configuración, cada endpoint tiene una sección `aor` (Address of Record) con el parámetro `max_contacts = 1`

Esto indica que solo un dispositivo puede estar registrado simultáneamente con ese identificador (sip1 o sip2). Si otro dispositivo intenta registrarse con el mismo usuario, el parámetro `remove_existing=yes` provocará que el contacto anterior sea eliminado para dar paso al nuevo.

## Ejercicio 2: Configurar los buzones de voz para sip1 y sip2

Usando como modelo el código 2, configura los buzones de voz para los clientes SIP **sip1** y **sip2** editando el fichero **voicemail.conf** de Asterisk siguiendo las siguientes instrucciones:

1. Fíjate que en la configuración de los clientes SIP (código 1) se indicaron sus correos en los parámetros **mailboxes** como:

- **001@sm** para **sip1**
- **002@sm** para **sip2**

Por lo tanto, configura los buzones de voz con número **001** y **002** en un contexto con el nombre **sm**.

2. Establece las siguientes opciones para los dos buzones como se indica:

- **nombre**: pon tu nombre.
- **email address**: pon el correspondiente email de la UDC.
- **short email address**: deja esta opción vacía.

3. Una vez configurados los buzones de voz asegúrate de recargar la configuración en Asterisk ejecutando en la consola de Asterisk el comando **voicemail reload**. A continuación ejecuta en la consola de Asterisk el comando **voicemail show users** para mostrar los buzones de voz configurados y **copia la salida obtenida**.

1 y 2 .-

```
1 [general]
2 format=wav49|wav
3 serveremail=voicemail@udc.test.es
4 attach=yes
5 maxsilence=10
6 maxlogins=3
7
8 [zonemessages]
9 central=America/Chicago|'vm-received' Q 'digits/at' IMP
10 european=Europe/Copenhagen|'vm-received' a d b 'digits/at' HM
11
12 [sm]
13 001 => 0000,Ruben,ruben.gonzalez.ouzounis@udc.es,,
14 002 => 0000,Ruben,ruben.gonzalez.ouzounis@udc.es,,
15
```

3-

```
freepbx*CLI> voicemail reload
Reloading voicemail configuration...
freepbx*CLI> voicemail show users
Context  Mbox  User              Zone      NewMsg
sm        001   Ruben              0
sm        002   Ruben              0
2 voicemail users configured.
freepbx*CLI>
```

### Ejercicio 3: Implementar la subrutina **sub\_recordcallinfo**

Implementa la subrutina **sub\_recordcallinfo** a partir del código 3 como sigue:

1. Completa el fragmento resaltado (a) con el resto de posibles valores de **DIALSTATUS**.
2. Completa el fragmento resaltado (b) con el código para crear una variable local llamada **familykey**, que contendrá la cadena *familia/clave* para AstDB, con el valor **calls\_usuario/clave**, siendo **usuario** el indicado en el parámetro **ARG1** de la subrutina y **clave** la establecida en la variable local **key**.
3. Completa el fragmento resaltado (c) con el código para asignar en AstDB el valor de **count + 1** a la familia/clave contenida en la variable local **familykey**.

1-

```
same => n,ExecIf($[${ARG2}=BUSY]?Set(LOCAL(key)=b))  
same => n,ExecIf($[${ARG2}=CHANUNAVAIL]?Set(LOCAL(key)=u))  
same => n,ExecIf($[${ARG2}=NOANSWER]?Set(LOCAL(key)=n))
```

2-

```
same => n,Set(LOCAL(familykey)=calls_${ARG1}/${LOCAL(key)})
```

3-

```
same => n,Set(DB(${LOCAL(familykey)})=${LOCAL(count)} + 1)
```

#### Ejercicio 4: Implementar la subrutina `sub_sipdial`

Implementa la subrutina `sub_sipdial` a partir del código 4 como sigue:

1. En el fragmento resaltado (a) usa la aplicación `Dial()` para llamar al *endpoint* indicado en el parámetro `ARG1`, con el *timeout* indicado en el parámetro `ARG2`, y las opciones indicadas en el parámetro `ARG3`.
2. Si la llamada realizada por la aplicación `Dial()` definida en el fragmento (a) no se responde (por *timeout* o por alguna otra razón) se continuará con el *dial-plan* en la extensión actual. En el fragmento resaltado (b), llama a la subrutina `sub_recordcallinfo`, pasándole como parámetros el *endpoint* indicado en el parámetro `ARG1` y el estado de la llamada indicado en `DIALSTATUS`.
3. Si la llamada se contesta, al finalizarla se saltará a la extensión especial *h*. Completa el fragmento (c) para que, cuando esa extensión se ejecute, se salte a la prioridad etiquetada como *record*, y que así se guarde en la base de datos también las veces que se contestó la llamada.

1-

```
same => n,Dial(PJSIP/${ARG1},${ARG2},${ARG3})
```

2-

```
same => n(record),GoSub(sub_recordcallinfo,start,1(${ARG1},${DIALSTATUS}))
```

3-

```
exten => h,1,Goto(start,record)
```

### Ejercicio 5: Implementar las extensiones 01 y 02

Implementa las extensiones 01 y 02 a partir del código 5. Para ello completa los fragmentos resaltados (a) y (b) con llamadas a la subrutina `sub_sipdial`, pasándole como parámetro el nombre del *endpoint* *PJSIP* correspondiente (`sip1` o `sip2`).

Una vez implementadas las extensiones y aplicados los cambios en Asterisk realiza algunas llamadas de prueba y comprueba que la información de las llamadas se registra en AstDB como se indicó en el apartado 2.3.1. Para ello ejecuta en la consola de Asterisk los comandos `database show calls_sip1` y `database show calls_sip2` y copia su salida. ¿Se registra correctamente la información de las llamadas?

a-  
same => n,GoSub(sub\_sipdial,start,1(sip1,10,))

b-  
same => n,GoSub(sub\_sipdial,start,1(sip2,10,))

Sí, se registra correctamente la información de las llamadas

```
Executing 102practical1.31 hangup< 10311/sip1 00000011 ,
== Spawn extension (practical1, 02, 3) exited non-zero on 'PJSIP/
freepbx*CLI> database show calls_sip2
/calls_sip2/a : 2
/calls_sip2/n : 1
/calls_sip2/o : 1
/calls_sip2/u : 3
4 results found.
freepbx*CLI>
```

### Ejercicio 6: Implementar la subrutina `sub_voicemail`

Implementa la subrutina `sub_voicemail` a partir del código 6 completando los fragmentos resaltados (a) y (b) como sigue:

1. En el fragmento resaltado (a) usa la función `IF` para establecer la variable local `message` a `b` si el parámetro `ARG2` (el valor de `DIALSTATUS`) es igual a la cadena `BUSY`, o establecerla a `u` en caso contrario.
2. En el fragmento resaltado (b) usa la aplicación `VoiceMail()` para lanzar el buzón de voz, pasándole como primer parámetro el valor de `ARG1`, y como segundo parámetro la concatenación de `message` y `ARG3`.

1.-

```
Set(LOCAL(message)={IF("${ARG2}"="BUSY"?b:u)})
```

2.-

```
VoiceMail(${ARG1},${message}${ARG3})
```

### Ejercicio 7: Implementar la extensión 00X

Implementa la extensión 00X. Para ello en primer lugar copia al *dialplan* las variables globales indicadas en el código 7, las cuales serán necesarias para el código de la extensión. A continuación implementa la extensión a partir del código 8 completando los fragmentos resaltados (a), (b), y (c) como sigue:

1. El el fragmento resaltado (a) usa un patrón de extensión que coincida con las extensiones 001 o 002.
2. En el fragmento resaltado (b) usa la variable de Asterisk que proporciona el número de la extensión actual para que este se reproduzca.
3. En el fragmento resaltado (c) inserta la expresión adecuada para que a la variable `user` se le asigne la cadena `sip1` si  $X = 1$ , o `sip2` si  $X = 2$ . Para esto simplemente concatena la cadena `sip` a la cadena obtenida de manipular la variable de Asterisk de la extensión actual para extraer su último dígito.

Una vez configurada la extensión prueba a dejar algún mensaje en el buzón de voz, luego ejecuta en la consola de Asterisk el comando `voicemail show users` y copia su salida. En la salida de este comando se indica para cada usuario en la columna «NewMsg» el número de mensajes nuevos en el buzón de voz. ¿Se están registrando correctamente los mensajes en el buzón de voz?

1.-

```
exten => 00[1-2],1,Verbose(1,Say extension and dial user)
```

2.-

```
same => n,SayDigits(${EXTEN})
```

3.-

```
same => n,Set(user=sip${EXTEN:-1})
```

Sí, se registran correctamente los mensajes en el buzón de voz

```
freepbx*CLI> voicemail show users
Context    Mbox  User           Zone      NewMsg
sm          001   Ismael         1         1
sm          002   Ismael         1         0
2 voicemail users configured.
```



### Ejercicio 8: Implementar la extensión 120

Implementa la extensión 120 a partir del código 9 completando los fragmentos resaltados (a), (b), y (c) como sigue:

1. En primer lugar, necesitamos obtener el nombre del *endpoint* que ha realizado la llamada, que en nuestro caso será `sip1` o `sip2`. Para esto usaremos la función `CHANNEL()`, con la que podemos obtener información del canal. En la sección 3.5 del documento *Notas de programación en Asterisk* tienes mas información acerca de esta función.
2. Una vez almacenado el nombre del *endpoint* en la variable `user`, en la siguiente prioridad de la extensión se guarda la dirección de buzón de voz en la variable `mailbox` a partir de la variable global correspondiente definida en el código 7. Sin embargo, si definimos un nuevo usuario SIP y no definimos una nueva variable global con su dirección de buzón de voz, el valor de la variable `mailbox` será la cadena vacía. Completa el fragmento resaltado (b) usando la aplicación `GotoIf()` de forma que si `mailbox` es una cadena vacía se salte a la prioridad con la etiqueta `end`. Para esto revisa el apéndice A.
3. Finalmente, Completa el fragmento resaltado (c) para consultar los mensajes del buzón de voz indicado en en la variable `mailbox`.

Una vez configurada la extensión el usuario podrá llamar y escuchar los mensajes que se le han dejado en su buzón. Prueba a llamar a la extensión, ¿Puedes escuchar los mensajes enviados al usuario?

1.-  
same => n,Set(user=\${CHANNEL(endpoint)})

2.-  
same => n,GotoIf("\${mailbox}" = "")?end

3.-  
same => n,VoiceMailMain(\${mailbox})

Se pueden escuchar los mensajes de cada usuario, los suyos propios, cuando llama al 120 con su correspondiente contraseña 0000.

### Ejercicio 9: Implementar la extensión 99X

Implementa la extensión 99X. Para ello primero copia al dialplan la variable global **TIMEZONE** indicada en el código 10. A continuación implementa la extensión a partir del código 11 completando los fragmentos resaltados (a), (b), y (c) como sigue:

1. El el fragmento resaltado (a) usa un patrón de extensión que coincida con las extensiones 991 o 992.
2. El el fragmento resaltado (b) usa la aplicación **GoToIfTime()** para comprobar si la llamada se produce en el horario permitido definido en la extensión 99X. En caso de que sea así, haz que se acepte saltando a la prioridad con la etiqueta **accept**.
3. Finalmente, en el fragmento resaltado (c) llama a la subrutina **sub\_sipdial** pasandole como usuario **sip1** si  $X = 1$ , o **sip2** si  $X = 2$ .

1.-

exten => \_99[1-2],1,Verbose(1,Accept calls only in the given times)

2.-

same => n,GoToIfTime(15:30-20:00,mon-thu,\*,\*,\${TIMEZONE}?accept)

3.-

same => n(accept),GoSub(sub\_sipdial,start,1(sip\${EXTEN:2},10))

### Ejercicio 10: Implementar la subrutina `sub_conference`

Implementa la subrutina `sub_conference` a partir del código 12 como sigue:

1. En el fragmento resaltado (a) usa la aplicación `Read()` para solicitar al usuario que introduzca un PIN de 4 dígitos y que suene el sonido `beep` antes de hacerlo. En la sección 3.4.3 del documento *Notas de programación en Asterisk* se explica en detalle el funcionamiento de `Read()`. Asegúrate de entender bien su funcionamiento antes de continuar.
2. En el fragmento resaltado (b) añade la lógica para verificar si el PIN introducido es correcto. Si es correcto, salta a la etiqueta `auth_success`, si no, reproduce un mensaje de error (`conf-invalidpin`) y vuelve al bucle de autenticación.
3. En el fragmento resaltado (c) haz que el usuario se conecte a la sala de conferencia número 7007 usando `ConfBridge()`.

1.-

```
same => n,Read(LOCAL(input_pin),beep,4)
```

2.-

```
same => n,GotoIf("${LOCAL(input_pin)}" = "${LOCAL(correct_pin)}"?auth_success)
```

```
same => n,Playback(conf-invalidpin)
```

```
same => n,Goto(auth_loop)
```

3.-

```
same => n,ConfBridge(7007)
```

### Ejercicio 11: Implementar la extensión 300

Implementa la extensión 300 a partir del código 13 completando el fragmento resaltado (a) como sigue:

1. En el fragmento resaltado (a) llama a la subrutina `sub_conference` pasándole como parámetros el PIN requerido (en este caso, 4323) y el número máximo de intentos (en este caso, 3).

1.-

same => n,GoSub(sub\_conference,start,1(4323,3))

### Ejercicio 12: Implementar la subrutina sub\_getcallscount

Implementa la subrutina `sub_getcallscount` a partir del código 14 como sigue:

1. En el fragmento resaltado (a) guarda en `LOCAL(familykey)` la ruta de la base de datos que almacena el valor pedido.
2. En el fragmento resaltado (b), usa la función `DB` para extraer el dato almacenado en la ruta `LOCAL(familykey)`.
3. Por último, en el fragmento resaltado (c) haz que la subrutina devuelva el dato. En la sección 3.4.5 del documento *Notas de programación en Asterisk* se explica el mecanismo por el que una subrutina puede devolver un resultado y como se obtiene este desde la extensión que hizo la llamada.

1.-

same => n,Set(LOCAL(familykey)=calls\_\${ARG1}/\${ARG2})

2.-

same => n,Set(LOCAL(count)=\${DB(\${LOCAL(familykey)})})

3.-

same => n,Return(\${LOCAL(count)})

### Ejercicio 13: Implementar la extensión #0

Implementa todos los casos descritos en la definición de la extensión #0. Revisa que la extensión ejecute las acciones correspondientes a cada situación (extensión inválida, se agota el tiempo de espera, etc.). Para ayudarte con el ejercicio, revisa el apéndice B, en donde se explica el manejo de los menús. Puedes también guiarte por el menú de ejemplo que se ve en el código código 15, pero recuerda adaptarlo a lo que se pide para #0.

```
[menu1]
exten => start,1,Verbose(1,Starting #0 menu)
same => n,Set(user=${CHANNEL(endpoint)})
same => n,Verbose(1,User detected as: ${user})
same => n,Background(press)
same => n,Background(digits/0)
same => n,Background(or-press)
same => n,Background(digits/1)
same => n,Background(or-press)
same => n,Background(digits/2)
same => n,Background(or-press)
same => n,Background(digits/3)
same => n,WaitExten(10)

exten => 0,1,Verbose(1,User ${user} requested unavailable calls)
same => n,GoSub(sub_getcallscount,start,1(${user},u))
same => n,Playback(vm-youhave)
same => n,SayNumber(${GOSUB_RETVAL})
same => n,Playback(calls)
same => n,Goto(menu1,start,1)

exten => 1,1,Verbose(1,User ${user} requested busy calls)
same => n,GoSub(sub_getcallscount,start,1(${user},b))
same => n,Playback(vm-youhave)
same => n,SayNumber(${GOSUB_RETVAL})
same => n,Playback(calls)
same => n,Goto(menu1,start,1)

exten => 2,1,Verbose(1,User ${user} requested answered calls)
same => n,GoSub(sub_getcallscount,start,1(${user},a))
same => n,Playback(vm-youhave)
same => n,SayNumber(${GOSUB_RETVAL})
same => n,Playback(calls)
same => n,Goto(menu1,start,1)

exten => 3,1,Playback(goodbye)
same => n,Hangup()

exten => i,1,Playback(pbx-invalid)
same => n,Goto(menu1,start,1)

exten => t,1,Playback(goodbye)
```

same => n, Hangup()

Y se añade en la subrutina practica1 la extensión:

[practica1]

exten => #0,1,Verbose(1,Entering call info menu)

same => n,Goto(menu1,start,1)

same => n, Hangup()